Architecture

Architecture

The first implementation slice follows `../plan.js.txt`:

- `src/runtime/index.ts` detects Node, Deno, or Bun and returns a fresh `GciRuntime` wrapper for each default `Session.connect()` call.
- `src/runtime/node.ts` calls `@gemstone-js/native`, the napi-rs addon in `../gemstone-js-native`. The native module import is cached, but the Node GCI wrapper is per session. The default backend uses the raw synchronous `Gci` object; `nativeSessionWorker: true` or `GS_NATIVE_SESSION_WORKER=1` selects `createGciSessionWorker()` so each session's native calls are queued through a dedicated worker-thread wrapper. Worker dispatch is strict: if the worker object does not expose the expected native operations, runtime initialization fails and closes the incomplete worker instead of falling back to the raw module and bypassing the session-thread boundary.
- `src/runtime/deno.ts` and `src/runtime/bun.ts` define the same low-level GCI surface through native FFI. Pointer-array and out-parameter calls use typed arrays so `perform()`, `fetchBytes()`, float conversion, `GciErr`, and dictionary lookups share the same runtime contract as the Node adapter. Shared buffer helpers live in `src/runtime/ffi-buffers.ts`.
- `src/runtime/library-discovery.ts` mirrors the Rust/Python loader order: explicit path, `GS_LIB_PATH`, scan `GS_LIB`, then scan `GEMSTONE/lib`.
- `src/client.ts` owns the public async `Session` API. This lets the native implementation keep blocking GCI calls behind a selectable backend without changing user code.
- `Session.executeObject()`/`evalObject()` and `executeManaged()`/`evalManaged()` mirror gemstone-py's typed/managed execute helpers while keeping raw `execute()` and value-marshalling `eval()` explicit.
- `Session.marshalOop()` mirrors the Python binding's marshalling order: immediate values, float conversion, string/symbol class detection, then raw OOP fallback.
- `Session.arrayOopToValues()` recursively reads GemStone `Array` instances through `size` and `at:` with cycle detection plus optional per-array, total-item, and depth limits for callers that need bounded readback.
- `Session.arrayOopToOops()` and `arrayObjects()` read raw and retained object handles from GemStone `Array` instances through the same `size`/`at:` path, with a `maxItems` bound for callers that need handles instead of values. `arrayOopToObjects()` is the direct retained-handle counterpart for callers that already have a raw Array OOP.
- `Session.arraySize()`, `arrayAt*()`, nullable `arrayFirst*()`/`arrayLast*()`, bounded `arrayPage*()`/`arrayTake*()`, batch `arrayPick*()`/`arraySetAll*()`, and single-index `arraySet*()` expose direct one-based GemStone `Array` metadata and indexed element access without materializing whole arrays.
- `Session.dictionaryOopToObject()` and `dictionaryValues()` read GemStone `StringKeyValueDictionary` instances through the `GsDict` key-enumeration path, preserving the existing string-key and value-marshalling behavior. Dictionary key, entry, item, and value-list readback accepts a `maxEntries` bound through `KeyedReadbackOptions`; `DictionaryReadbackOptions` remains a compatibility alias for dictionary callers. Session-level dictionary key/item/value-list, get/set/replace/remove, pick/require, and size helpers delegate to the same wrapper path for callers that have a raw dictionary OOP but do not need to keep a `GsDict` wrapper around.
- `Session.argumentToOop()` handles the common JS-to-GemStone path. Use `perform()` for raw OOP arguments; use `performWith()` when you want JS values converted into GemStone objects.
- `Session.bulkPerformOop()` and `bulkPerformCallsOop()` mirror gemstone-py's batched selector-send helpers. They render one Smalltalk eval that sends a shared selector across many receivers or a mixed list of receiver/selector calls, then parse newline-delimited `asOop` results back into branded OOPs. The

`bulkPerformValue()` and `bulkPerformCallsValue()` variants marshal each returned OOP through the normal session marshalling path.

- `Session.bulkPerformWith()` and `bulkPerformCallsWithOop()` add the same batching shape for callers that want JavaScript argument marshalling first. Arguments are converted through `argumentToOop()`/`argumentsToOops()` once before the generated Smalltalk batch source is rendered.
- `bulkPerformObjects()`, `bulkPerformObjectsWith()`, `bulkPerformCallsObjects()`, and `bulkPerformCallsObjectsWit` retain each returned OOP as a `TypedOop<T>`, matching the single-send `performObjectWith()` contract for batched object workflows.
- `Session.classRef()` is the first explicit object-model layer: it caches class symbol resolution and exposes async class-side sends, object-returning sends, and allocation while keeping remote calls visible. Class names use the shared simple GemStone global-name validation policy because classes live in globals.
- `GsDict` and `PersistentRoot` are convenience layers over the same session primitives. They should stay thin: the session owns marshalling and GCI calls, while wrappers expose explicit value/raw/object accessors, value-named setter aliases, object-named setter aliases for stored OOP handles, nested dictionary helpers including batch dictionary setters, send helpers, dictionary metadata, dictionary/root enumeration, replace/clear lifecycle helpers for owned string-key dictionaries, bounded dictionary/global/root enumeration through `KeyedReadbackOptions`, global/root size helpers, and required global/root accessors. `PersistentRoot.userGlobals()`, `globals()`, `published()`, and `sessionMethods()` mirror the named SymbolDictionary constructors in gemstone-py.
- `OrderedCollection` mirrors gemstone-py's plain GemStone ordered sequence wrapper while keeping JavaScript's async style explicit. It wraps an existing OOP or creates `OrderedCollection new`, exposes value/raw/object accessors, uses zero-based indexed reads with negative indexes, and keeps mutation visible as direct sends such as `add:`, `addAll:`, `removeFirst`, and `removeLast`.
- `GStore` builds on `PersistentRoot` and `GsDict` rather than adding another storage layer. Each named store is a `StringKeyValueDictionary` under `UserGlobals.GStoreRoot`; values are JSON strings, transaction callbacks use an in-memory snapshot plus dirty/delete buffers, `GStore.list()`, `read()`, existence checks, and transaction snapshot loading accept `KeyedReadbackOptions` bounds, and the caller's session owns transaction visibility.
- Migration helpers are intentionally library-first. Version metadata and advisory locks are JSON strings in `UserGlobals`, so the runner can use the existing root marshalling path and stay reviewable. The public API validates dependency order, unknown applied ids, and checksum drift before applying steps; each step commits after its metadata write.
- Transaction retry helpers are intentionally callback-based so the whole unit of work can be replayed after a commit conflict. Existing sessions are aborted and reused between attempts; owned sessions are recreated per attempt. `commitWithConflictDetails()` converts conflict-like commit failures into a structured `CommitConflictError` by reading `System conflictReportString` and the current transaction conflict collections when GemStone exposes them. `nestedTransaction()` mirrors gemstone-py's nested transaction helper with explicit begin/commit/abort sends and preserves structured conflict errors on failed nested commits.
- Benchmark helpers are split between report generation and saved-artifact policy. `src/benchmarks.ts` can generate compact reports from the offline `gci` suite or opt-in live persistence suites, while `src/benchmark-baselines.ts` validates saved report JSON, compares result rows with regression thresholds, selects metadata-compatible baselines from a manifest, can compare a candidate directly against the selected manifest baseline, rejects ambiguous duplicate baseline metadata by default, can intentionally replace matching manifest entries during registration, and updates baseline manifests for CI enforcement.
- `RcCounter`, `RcKeyValueDictionary`, and `RcQueue` are thin wrappers over the GemStone reduced-conflict classes used by gemstone-py. They keep creation, session factory helpers, sends, enumeration, and raw-OOP variants explicit so callers can choose value marshalling or handle-level APIs without hiding transaction behavior.
- `bootstrapGemStone()` and `gemstone-js-bootstrap` mirror the useful GemStone-side bootstrap flow from `gemstone-py`: audit known helper roots, create missing roots idempotently, and write a JavaScript-specific bootstrap version marker while leaving application data intact.
- Source-rendered helper names and class-ref names share one validation policy. Collection names, class names, persistent-root names, persistent-root entries, and direct global names must be simple GemStone global-style identifiers; dictionary string keys are passed through string-key GCI APIs and are not constrained by that policy. See `docs/naming.md`.
- `ObjectLog` is a session-bound wrapper over GemStone `ObjectLogEntry`. It reads batch ObjectLog rows

into one escaped string payload, supports `maxEntries` sentinel bounds plus `{ level, order }` through `ObjectLogReadOptions`, can fetch the newest tail without scanning from the first row, filters requested levels on the GemStone side, exposes count and presence checks without fetching rows, offers deterministic summary/format helpers for operator output, supports level-scoped clear and descending-index bulk deletion, preserves real log indexes, and parses locally, matching the `gemstone-py` parser contract while keeping commits and aborts explicit through the caller's session.

- Query helpers render simple selector paths, expose collection metadata, can count/check predicate matches without materializing selected results, can read whole collections, bounded pages, indexed items, or collection endpoints as handles or marshalled values, can check, add, remove, replace, or clear collection members through explicit value and raw-OOP helpers, and delegate GemStone array unwrapping to the session readback helpers so iterators yield object handles instead of chunk containers. Selector paths and comparison operators are both validated before source is emitted, and alias helpers such as `find()`, `any()`, and `none()` use the same underlying bounded query forms as `first()` and `exists()`.
- Codegen helpers validate generated JavaScript identifiers and emit wrappers that choose between `performValueWith()`, `performWith()`, or `classRef().sendObject()` based on the requested return kind. Selector shape, arity, manifest structure, and module export uniqueness are checked before source is emitted so manifest mistakes fail locally.
- Generated modules can include type-only/value imports plus typed session, argument, and return annotations. The JSON Schema in `schemas/` gives editor feedback for hand-written manifests.
- The `codegen` npm script is a manifest-driven file generator around the same renderer, with a `--check` mode for keeping generated wrapper source explicit, reviewable, and current in CI.
- The `codegen:scan` script is a small source scanner for decorated classes. It emits reviewable manifests by default and can emit generated wrapper modules directly with `--module`; either mode can be checked against a committed file with `--check --out`.
- The planned object-mapping layer should build on the same reviewable codegen discipline. The runtime now exposes `mappedObject()` for opt-in async property-style selector sends over a retained handle, with explicit selector, setter, object-return, raw-OOP, and snapshot policy. It also exposes `transparentObject()` for awaitable selector properties, queued assignment writes, optional per-proxy caching, and request-scoped identity reuse through `TransparentObjectMapper`. The next layer is a mapping manifest schema for GemStone class names, TypeScript types, selectors, setters, repository selectors, lifetime rules, and snapshot fields; generated `*Ref` classes that wrap `TypedOop<T>` or delegate to the runtime mapping helpers; repository helpers that return typed refs; and bounded snapshot/dictionary helpers for UI/API payloads. Explorer and VS Code mapping views should read committed mapping manifests and generated files rather than hiding an automatic runtime mapper.
- `scripts/check-public-surface.mjs` parses `src/index.ts` with the TypeScript compiler API and compares value/type exports, source modules, and aliases against `scripts/public-surface.expected.json`. This keeps the public barrel explicit and makes API changes reviewable.
- `gemstone-js-api-contract` imports the package self-reference and compares runtime value exports, package metadata, schema exports, and CLI bin targets against the committed public-surface contract. The checker deliberately does not create a session, so it can run against local or installed artifacts without `@gemstone-js/native`.
- `Session.inspect()`, bounded recursive `dump()`, and `describeClass()` ask GemStone for compact string payloads and parse them locally, avoiding a dependency on dictionary marshalling for debug metadata. `Session.printString()`, `GemStoneClassRef.describe()`, and retained handle `inspect()`/`dump()`/`printString()` expose the same debug path from raw handles, class refs, and retained handles.
- `TransactionScope` and `RequestScope` mirror gemstone-py's framework-neutral web core in async JavaScript form. They centralize request failure detection, commit-on-success, abort-on-exit/error/status, clean pool release, and owned session logout so framework adapters can share lifecycle policy instead of duplicating transaction teardown logic.
- Express, Fastify, and Hono adapters delegate teardown through `RequestScope`, attach the active scope to the request/context, default to abort-on-4xx semantics, and expose `transactionPolicy`/`serverErrorStatus` options for framework-specific policy tuning. `docs/framework-adapters.md` and the `examples/web-*.ts` files show the shared pool, health-check, ObjectLog, and shutdown patterns.
- `SessionPool.warm()` targets total pool capacity and `stats()` includes pending acquires, recycle discards, idle-timeout discards, and validation failures so saturated pools can be observed. `maxSessionAgeMs`,

`maxSessionUses`, and custom `healthCheck` callbacks provide the same production recycling hooks as gemstone-py's session providers. `snapshot()` and `eventListener` expose provider-style capacity snapshots and lifecycle events, and pool events are also mirrored into the configured metrics/tracer hooks. `withSession()` gives applications a callback helper that always releases the lease and preserves callback errors if cleanup fails.

- `InMemoryMetrics` and `InMemoryTracer` provide dependency-free observability recorders for tests while the OpenTelemetry adapter maps to real spans.
- `src/runtime/serialized.ts` serializes all session-bound calls and reactivates the session id before dispatching into GCI. This is the current safety layer before a dedicated native session thread lands.
- `ValueConverterRegistry` is an opt-in layer above default argument marshalling. A session copies the configured registry, checks converters before falling back to built-in array/dictionary/object handling, and can round-trip converter-selected OOPs by converter name. The built-in scalar registry starts with ISO-string `Date` support, matching gemstone-py's explicit converter model without changing default persistence semantics. `objectToDictionaryArgument()` provides the same explicit boundary as gemstone-py's `dataclass_to_dict()`: class instances become plain dictionary payloads only when the caller opts in, with recursive conversion for nested class instances and arrays.
- `src/testing/mock-runtime.ts` lets the high-level API be tested without a live GemStone instance.

The runtime contract deliberately mirrors the function list in `gemstone-py/gemstone_py/_gci_ctypes.py` and `gemstone-rs/crates/gemstone-gci/src/lib.rs`.